ASSIGNMENT 13

Reverse Engineer an Android APK Using JADX or APKTool and Analyze the Application Source Code

1. Introduction

Android applications are distributed as APK (Android Package Kit) files. Reverse engineering is the process of examining an application package to understand its structure, resources, manifest, and program logic. Tools such as JADX and APKTool help security analysts inspect an APK for learning, debugging, and authorized security assessment. JADX converts DEX bytecode into readable Java-like source, while APKTool decodes resources and the Android manifest.

2. Objectives

- Understand the structure of an Android APK.
- Use JADX or APKTool to inspect an authorized APK.
- Review the manifest, permissions, components, and decompiled code.
- Identify potential security weaknesses and recommend improvements.

3. Requirements

- Computer with Java installed, as required by the selected tool version.
- JADX GUI or JADX command-line tool, and/or APKTool.
- An APK that you created or are explicitly authorized to analyze.
- A folder to store the extracted analysis files.

Safety note: Analyze only APKs you own or have permission to inspect. Do not use reverse engineering to steal credentials, bypass licensing, or modify third-party applications for unauthorized use.

4. Method 1: Analyze an APK Using JADX

JADX is useful for reading Java-like representations of Android DEX bytecode. Download it from the official JADX GitHub project and launch jadx-gui, then open the authorized APK.

Alternatively, use the command line:

```
jadx -d jadx_output app.apk
```

Replace app.apk with the path to the authorized APK. The output folder contains reconstructed source files and resources where decompilation is possible. Decompiled output may not exactly match the original source code, especially when the app is obfuscated.

5. Method 2: Decode an APK Using APKTool

APKTool is useful for decoding the Android manifest and resources and for examining smali code, which is a readable assembly-like representation of DEX bytecode. Install APKTool using its official instructions, then run:

```
apktool d app.apk -o apktool_output
```

Open the output folder and review AndroidManifest.xml, res/ resources, and smali/ code. APKTool does not generally reconstruct the original Java source; use JADX when readable Java-like code is required.

6. Analyze the Application Structure

Start with the AndroidManifest.xml file. Record the package name, minimum and target SDK versions where available, permissions, and declared Activities, Services, Broadcast Receivers, and Content Providers. Review exported components and determine whether external access is required and appropriately protected.

Next, inspect the decompiled source or smali code. Search for network endpoints, file storage operations, authentication logic, cryptographic usage, WebView configuration, logging, and handling of sensitive information. Treat strings and code findings as leads that require context; a string alone does not prove a vulnerability.

7. Security Checks

- Permissions: Are sensitive permissions necessary for the application's function?
- Exported components: Are externally accessible Activities, Services, Receivers, or Providers protected?
- Secrets: Are API keys, passwords, or tokens embedded in the APK?
- Network security: Does the application use HTTPS and handle certificate validation appropriately?
- Data storage: Is sensitive information written to logs or stored in plain text?
- Input handling: Are Intent data, deep links, and WebView inputs validated?
- Obfuscation: Is the code obfuscated? Obfuscation may make analysis harder but is not a substitute for security controls.

8. Findings and Observation Table

Record only findings supported by the APK being examined. The following table is a template; do not assume a vulnerability exists before verifying it.

Area Checked	Observation / Evidence	Risk Level	Recommendation
Manifest and permissions	Record actual evidence	Low / Medium / High	Record suitable mitigation
Exported components	Record actual evidence	Low / Medium / High	Record suitable mitigation
Hard-coded secrets	Record actual evidence	Low / Medium / High	Record suitable

			mitigation
Network communication	Record actual evidence	Low / Medium / High	Record suitable mitigation
Data storage / logging	Record actual evidence	Low / Medium / High	Record suitable mitigation
Input validation	Record actual evidence	Low / Medium / High	Record suitable mitigation

9. Limitations

Static analysis examines an application without running it. Results may be incomplete because of obfuscation, dynamically loaded code, native libraries, or behavior that appears only at runtime. A suspected issue should be confirmed in an authorized test environment, and sensitive data should not be copied into the report.

10. Conclusion

JADX and APKTool are useful tools for examining Android APK structure and application logic. JADX provides Java-like decompiled code, while APKTool decodes resources, the manifest, and smali files. Reviewing permissions, exported components, embedded secrets, network handling, and data storage can reveal potential weaknesses. Findings should be supported by evidence, assigned an appropriate risk level, and paired with practical mitigations.

References

1. JADX GitHub project: https://github.com/skylot/jadx
2. APKTool official website: https://apktool.org/
3. Android Developers — App manifest overview: https://developer.android.com/guide/topics/manifest/manifest-intro